

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 7.1a: Why LLMs and Tokenization

Zhigang Feng

Spring 2026

Topic 7.1a Agenda

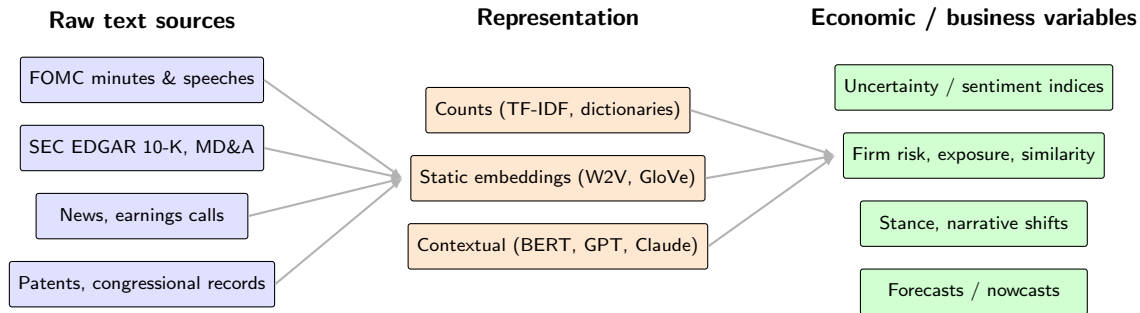
1. **Why LLMs? Motivation and History**
2. **Tokenization** — BPE, WordPiece, SentencePiece

Topic 7.1b continues with word embeddings (TF-IDF, Word2Vec, GloVe), geometry of meaning, and the bridge to contextual embeddings.

Arc: T1a Why+Tokens → T1b Embeddings → T2a Attention → T2b Transformer → T3a Training → T3b Limits → T4 Applications.

Why LLMs? Motivation and History

Text as Economic Data — The Big Picture



This lecture builds the middle column. T1b: counts → static embeddings. T2a/T2b: contextual embeddings. T3a: how those models are trained. T3b: when not to trust them. T4: the right-hand column.

Why Economists Care About LLMs

- **Text as data** — modern economic research increasingly leverages textual sources:
 - Corporate filings (10-K, 10-Q, MD&A)
 - Central bank speeches, FOMC minutes, ECB statements
 - Earnings calls, analyst reports, regulatory filings
 - News, social media, congressional records
- **Applications:**
 - Sentiment analysis for financial forecasting
 - Policy stance detection and uncertainty indices
 - New text-derived macroeconomic indicators
 - Summarization of long, technical documents
- **Why LLMs in particular:**
 - Capture nuanced language and domain-specific jargon
 - Handle long-range context (sentences, paragraphs, whole documents)
 - Provide a unified architecture for many tasks

Three Roles of Text in Economic Research

Role	What text is	Example
Measurement tool	A way to construct previously unobserved variables	Baker–Bloom–Davis (2016) Economic Policy Uncertainty index from newspapers
Behavioral outcome	A response variable revealing decisions or institutional change	Hansen, McMahon & Prat (2018): how FOMC transparency reform changed committee speech
Treatment variable	A quasi-experimental input affecting behavior	Central bank communication shocks and inflation expectations

Take-away: the same text can play very different econometric roles. Choose your representation method to match the role—transparency matters for measurement, while subtle context matters for behavior modeling.

A Brief History: RNN → Transformer → GPT

- **2003:** Bengio et al. neural language model — continuous word vectors
- **2013:** Mikolov et al. *Word2Vec* — dense word embeddings at scale
- **2014:** Pennington et al. *GloVe* — global co-occurrence matrices
- **2017:** Vaswani et al. “*Attention Is All You Need*” — the Transformer replaces RNNs/LSTMs
- **2018:** Devlin et al. *BERT* (encoder); Radford et al. *GPT* (decoder)
- **2020–2022:** GPT-3, instruction tuning, RLHF → ChatGPT moment
- **2023–2026:** GPT-4/4o, Claude (3, 4), Gemini, LLaMA-3, DeepSeek; long-context (100k–1M tokens), open-weights ecosystem, agentic systems

Driver of the shift: attention scales better than recurrence, both in compute (parallel) and in quality (long-range dependencies).

The Autoregressive Principle

- An LLM is fundamentally a **next-token predictor**:

$$p(t^{(n+1)} \mid t^{(1)}, t^{(2)}, \dots, t^{(n)}).$$

- **Generation:** sample $t^{(n+1)}$ from this distribution, append to the context, and repeat.
- **What this single equation enables** (we preview the intuition on the next two slides; a full revisit comes after we understand pretraining and fine-tuning):
 - **Summarizing 10-K filings or FOMC minutes** — prompt with document + instruction; the model predicts summary tokens one by one
 - **Generating narrative forecasts** conditional on macro state — condition on data in the prompt; next tokens extend the narrative
 - **Producing scenario analyses in plain language** — the scenario is the context; prediction fills in the implications
 - **Writing and debugging code** for empirical work — same objective over a code corpus yields code completions
- **Key insight:** “everything is next-token prediction” is mathematically tiny but pedagogically and practically enormous.

Intuition: Summarizing a 10-K Filing

- **Task:** condense 50 pages of an SEC 10-K risk section into 3 readable bullets.
- **How the autoregressive equation achieves this:**
 1. Concatenate the instruction and document into the prompt:
■ Summarize the following 10-K risk section in 3 bullets:\n\n[text]\n\nSummary:■
 2. The model predicts token $t^{(n+1)}$ from the full prompt; this becomes the first word of the summary
 3. Each subsequent token is predicted given everything before it (including the already-generated summary words)
 4. Generation stops at a period or end-of-sequence marker
- **Why does it work?** During pretraining the model read millions of (long document, short summary) pairs. The next-token distribution has implicitly learned: *“after an instruction like this, produce a concise factual summary.”*
- **Economic punchline:** a 50-page MD&A section is compressed into 3 bullets in under one second — and the same pipeline scales across thousands of filings.

Intuition: Extracting Stance from FOMC Minutes

- **Task:** classify each sentence in FOMC minutes as *hawkish*, *dovish*, or *neutral*.
- **How the autoregressive equation achieves this:**
 1. Prompt: ■Sentence: [sentence]\nStance (hawkish/dovish/neutral):■
 2. The model predicts the next token; because the vocabulary contains these exact words, the top-probability token *is* the label
 3. With fine-tuning on labeled FOMC data, the next-token distribution concentrates on monetary-policy-relevant labels
- **What fine-tuning changes:** the base model already understands “hawkish” and “dovish”. Fine-tuning shifts the distribution so that, after a central-bank sentence, stance-label tokens are consistently most probable.
- **Economic application:** automated stance extraction across decades of FOMC minutes — enabling the kind of large-scale communication analysis in Hansen, McMahon & Prat (2018) without manual coding.

An Identification-First View of Text

- **Engineering perspective first:** to a machine, a document is just a character string. But every string was *produced* — by a person, in a situation, for an audience. Before any econometrics, ask: what generated this string?

- **The data-generating process (DGP):**

$$x_i = g(s_i, a_i, c_i) + \varepsilon_i$$

- s_i : the latent **economic state** we want to measure (policy stance, sentiment, firm risk)
- a_i : the speaker's **strategic framing** — audience management, spin, emphasis, tone chosen for effect
- c_i : **institutional context** — venue, legal disclosure requirements, format conventions
- **Why this matters for a downstream regression** $y_i = \beta m(x_i) + \gamma z_i + u_i$:
 - If framing a_i is correlated with the error u_i , the text feature $m(x_i)$ is *endogenous* — the same omitted-variable problem as always, but hidden inside the language
 - Measurement error is non-classical: it correlates with the true state s_i , biasing $\hat{\beta}$
- **Two research uses:** (a) **Measuring** s_i — choose representations that filter out a_i ; (b) **Studying strategic communication** — a_i is your outcome variable (e.g. Hansen, McMahon & Prat 2018).

Tokenization

Tokenization: The First Step

- **Goal:** convert raw character sequences into discrete *tokens* that a neural network can consume.
- **Why not just split by spaces?**
 - Many economically relevant terms combine words, numbers, and symbols (e.g. S&P500, CO2, Q4_2025, stagflation)
 - Punctuation and formatting carry meaning ('\$1.2B' vs '1.2 billion')
 - Different languages have different word boundaries (or none at all)
- **What is a token?** A flexible unit:
 - A whole word ('inflation')
 - A subword ('infl', 'ation')
 - A single character or byte
 - A special marker (<EOS>, <PAD>, <USER>)

Vocabulary and the Tokenization Map

- Let A be the alphabet (Unicode characters). A tokenizer is a function:

$$\mathcal{K}: A^* \longrightarrow \{1, 2, \dots, M\}^*$$

mapping any text string into a sequence of **numbers** — each number is a positive integer that serves as an index into a lookup table of size M . The fact that the index is an integer is incidental; what matters is that it *uniquely identifies* each token, the same way a ZIP code uniquely identifies a location.

- **Why integers?** Neural networks need numeric inputs; each token ID will index into an embedding lookup table.
- **Practical constraint — context length:**
 - Models have a maximum input size n (typically 2,048 to 128,000 tokens; some 1M)
 - Long policy documents, FOMC transcripts, or 10-K filings often exceed n
 - Forces *chunking*, *summarization*, or *retrieval* (a hint at Lec08)
- Vocabulary sizes are typically 30,000–100,000 subword tokens. **To put this in perspective:** Merriam-Webster Unabridged lists $\approx 470,000$ entries; a college-educated native speaker actively uses roughly 20,000–40,000 words. A subword vocabulary of $M = 50,000$ therefore spans roughly $\frac{470,000}{50,000} \approx 9.4$ words, or $\frac{40,000}{50,000} \approx 0.8$ words on average. This means that most tokens in the vocabulary represent only a fraction of the words in the language, but together they cover the entire range of possible words.

Illustrative Example: “GDP rose by 2%”

- A simple tokenization might produce

(GDP, rose, by, 2, %)

mapping to integer IDs, e.g. (541, 823, 71, 312, 49).

- **Why integer mapping matters:**

- Each ID becomes the index of a learned embedding vector
- Numeric tokens ('2', '3', '5') get *their own* embeddings

- **Injectivity matters for forecasting:** a well-designed tokenizer maps

- “inflation rate is 3.5%”
- “inflation rate is 3.6%”

to *different* token sequences. Otherwise small but economically meaningful differences would collapse into the same input.

- **Tokenizer/model coupling:** the tokenizer is part of the model. If you swap the tokenizer, all the embedding rows are meaningless — this is why every published LLM ships with its tokenizer.

Subword Tokenization: BPE, WordPiece, SentencePiece

Algorithm	How it learns the vocabulary	Where you see it	Notable feature
Byte-Pair Encoding (BPE)	Start from chars; greedily merge the most frequent adjacent pair until vocab cap is reached	GPT-2/3/4, LLaMA, Mistral	Byte-level variant handles any Unicode
WordPiece	Like BPE, but merge the pair maximizing corpus likelihood instead of raw frequency	BERT, DistilBERT	## prefix marks subword continuations
SentencePiece	Trains directly on raw text (no whitespace presplit); unigram or BPE objective	T5, mT5, LLaMA, XLM-R	Multilingual; treats space as a normal character

Why subwords? OOV handling (QuantitativeEasing $\rightarrow$ Quant+itative+Easing); one token for common words, splits for rare/compound; one vocab across languages.

Cost: nowcasting may split into 3 tokens; 1.2345 into several digit-tokens — making numerical reasoning fragile.

Tokenization Is a Preprocessing Choice

- Tokenization sits inside a larger **preprocessing pipeline**:

$$x^{\text{raw}} \xrightarrow{\text{clean}} x^{\text{clean}} \xrightarrow{\text{tokenize}} \{t_{ij}\} \xrightarrow{\text{represent}} z_i,$$

from raw scraped text $\rightarrow$ normalized text $\rightarrow$ integer token IDs $\rightarrow$ downstream numeric representation (TF-IDF, embedding, ...).

- **Specification curve problem** (Denny & Spirling, 2018): just as economists report results under alternative lag orders, control sets, and sample restrictions, NLP results should be reported under alternative preprocessing pipelines.
 - LDA topics under different pipelines vary substantially — often by as much as the substantive variation of interest
 - Stemming and punctuation handling have the largest individual effects
 - **Lesson:** the pipeline is a modeling decision; report the full range as you would a VAR robustness table
- **Lesson for empirical economists:** report the *universe* of preprocessing choices. The “convenient” specification is not a result — it is a choice. This applies equally to tokenizer choice and vocabulary size.

The Seven Preprocessing Choices

- **Lowercasing** — map Fed and fed to the same token. Compresses vocabulary, but erases proper-noun signal.
- **Stemming / lemmatization** — cut suffixes so *banking/banks/banked* collapse to *bank*. Can erase tense, plurality, policy-language nuance.
- **Stop-word removal** — drop high-frequency function words. Reduces noise; but negation (*not*) and modality (*may, will*) can matter for central-bank text.
- **Punctuation handling** — keep, strip, or tokenize separately. Percent signs, decimals, sentence boundaries all matter.
- ***n*-gram inclusion** — treat “*yield curve*” as one token rather than two. Captures multi-word economic concepts at the cost of a larger vocabulary.
- **Frequency cutoffs** — drop terms in $< k$ or $> K$ documents. Removes typos and boilerplate; can also remove rare-but-informative events.
- **Language filtering** — restrict to English or apply per-language pipelines. Crucial for multilingual corpora like ECB speeches.

Each choice is an independent on/off switch $\Rightarrow 2^7 = 128$ pipelines $\Rightarrow$ report a *specification curve*, not a single run.